AI Assisted Software Development Practice

Monorepo vs. Polyrepo

- Monorepo: Single Git repository containing code for multiple related projects
- Polyrepo (or multirepo): each project or service lives in its own separate repository

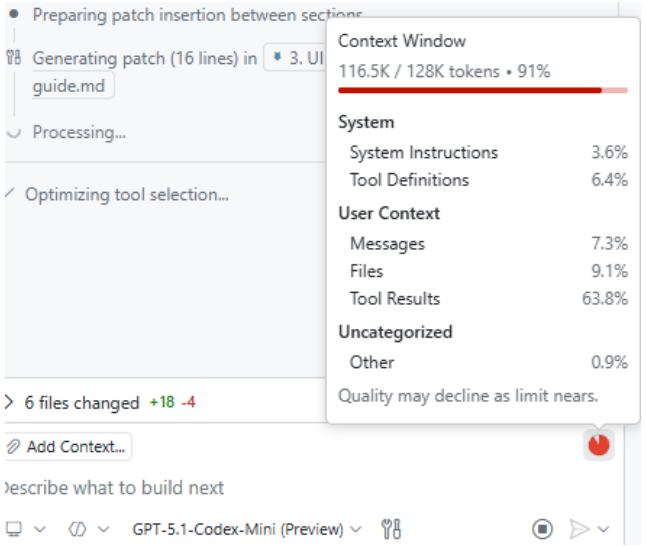
```
my-ecommerce/
├── apps/
│   ├── frontend/     # React app for UI
│   └── backend-api/  # Main Node.js API
├── services/
│   ├── products/     # Microservice for catalog
│   ├── orders/       # Microservice for order processing
│   ├── payments/     # Microservice for transactions
│   └── auth/         # Microservice for user auth
├── packages/
│   ├── shared-ui/    # Reusable React components
│   └── utils/        # Common utilities (e.g., validation)
├── package.json      # Root with workspaces
```

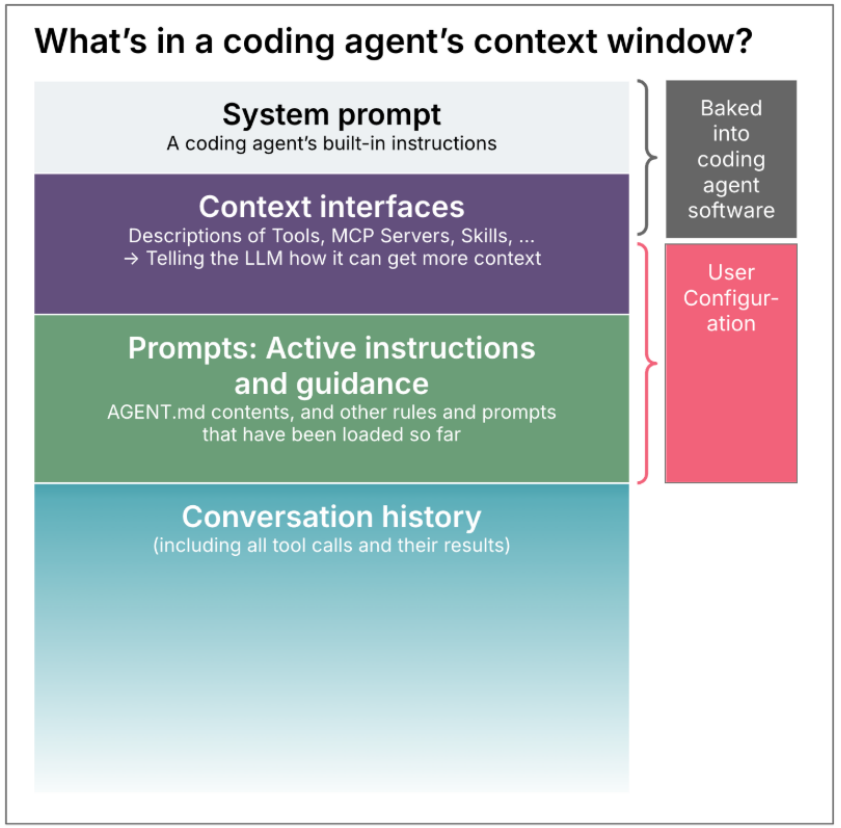
```
Separate Git repositories
•  ecommerce-frontend (React)
•  ecommerce-backend-api (Node.js)
•  ecommerce-products-service
•  ecommerce-orders-service
•  ecommerce-payments-service
•  ecommerce-auth-service
```

Which approach is better for AI-assisted coding?

Context windows for coding agents

- Conversation history: your messages and the agent's prior replies/edits.
- System & agent setup: built-in role prompt, safety rules, and project profile.
- Code & docs: relevant source files, README, and guidance files (e.g., CLAUDE.md, rules).
- Task instructions: the current coding request and any constraints or patterns.
- Tool descriptions: schemas or specs for available tools/MCP servers and plugins.
- Tool outputs: JSON results, errors, or logs from invoked tools.
- Streaming output: the agent's in-progress generated code or messages.

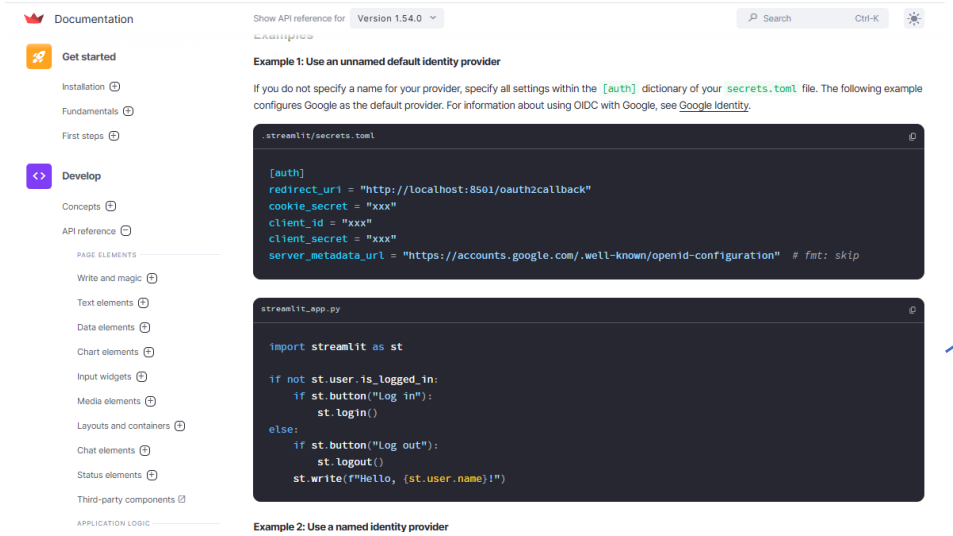


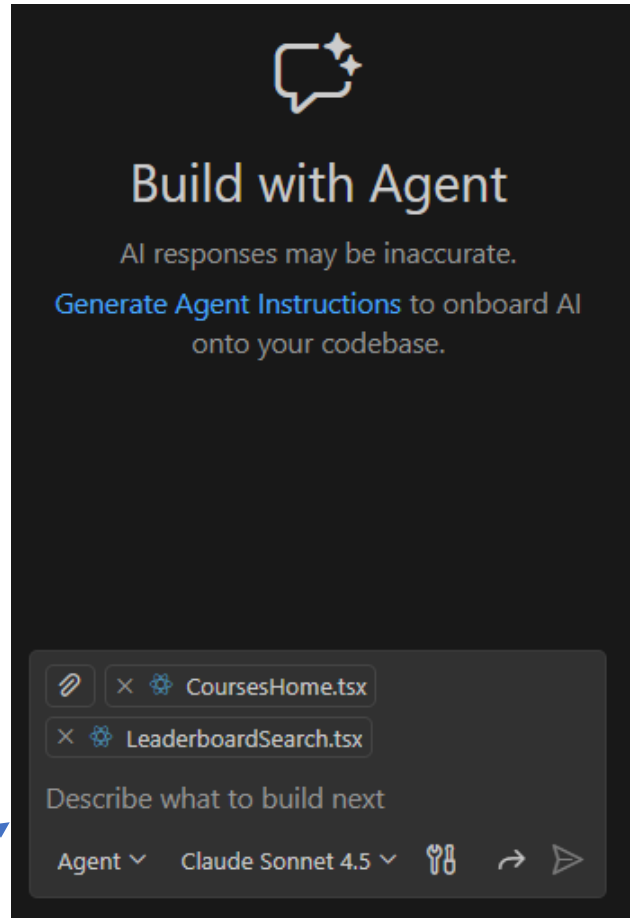


https://martinfowler.com/articles/exploring-gen-ai/context-engineering-coding-agents.html

Ground the model with right context

- Provide the right context (relevant files, contracts, examples) so the model doesn't guess
  - High-level goals, examples of good patterns, latest version of an API/library, niche or internal SDKs, etc
- Context engineering
  - Do not dump entire repositories into the prompt window.
  - Excluding irrelevant context to save costs and improve AI accuracy

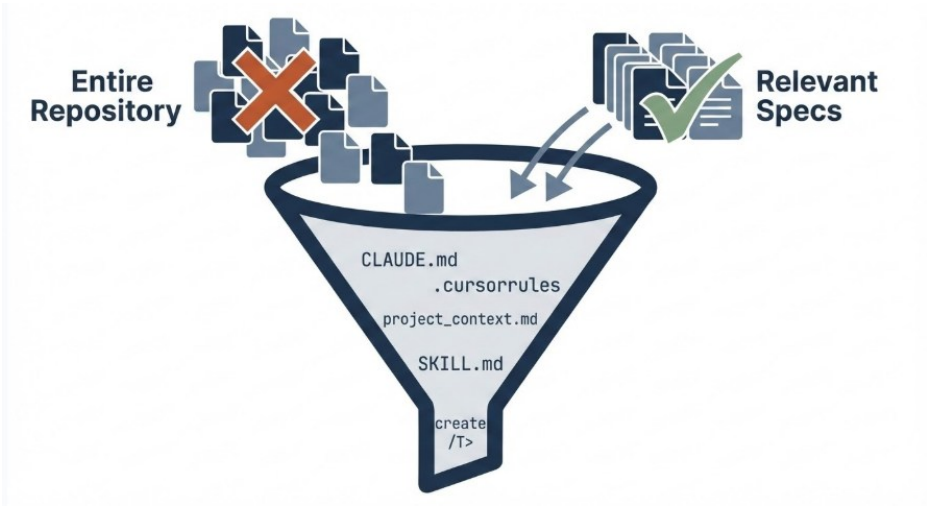




- Summarize the library/doc with examples
- More specific prompt:
  Create a streamlit app. Use st.login() for authentication

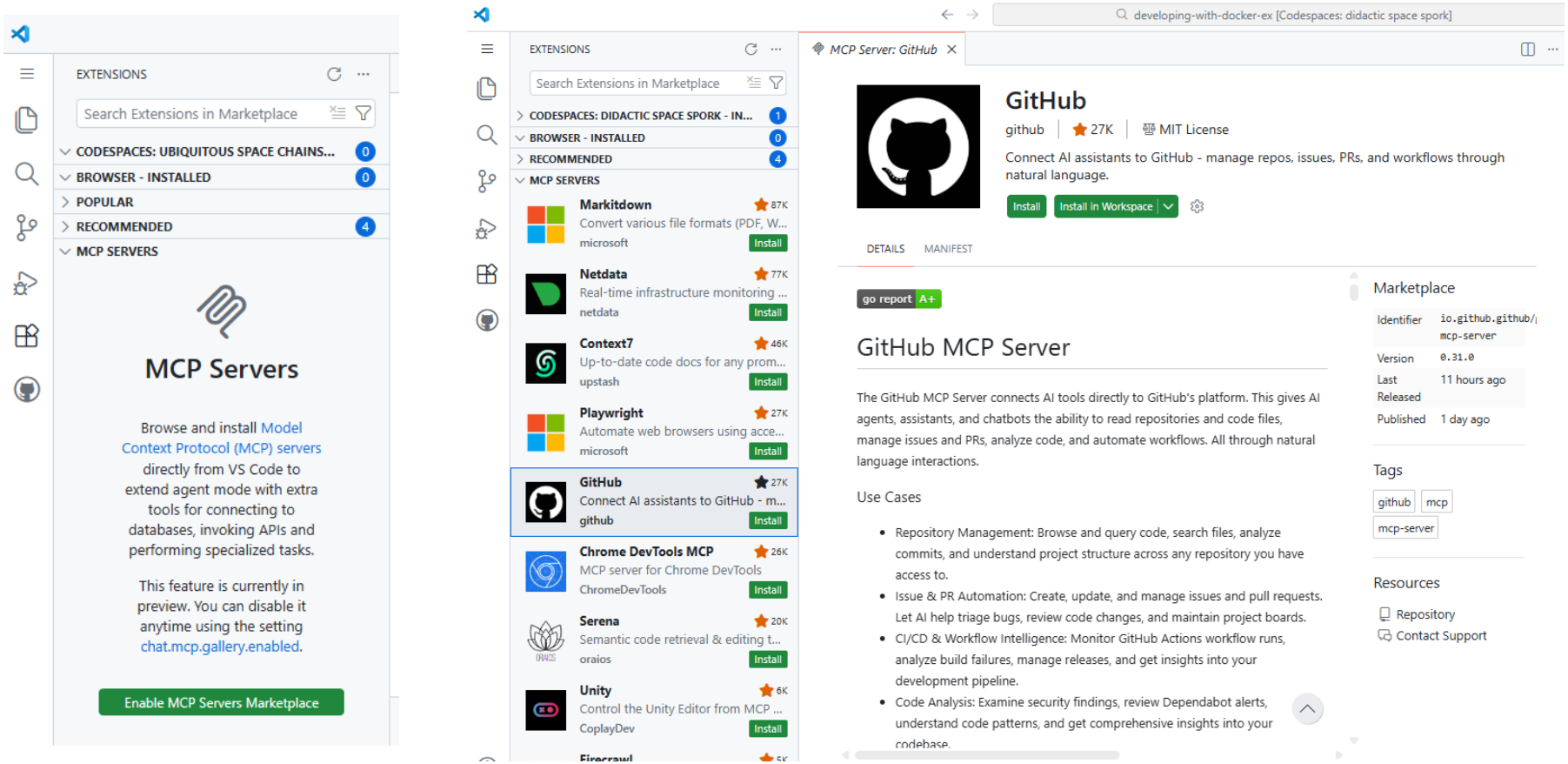
What is Context Engineering?

- Carefully curating what the LLM sees in a coding-agent session to improve quality, reliability, and alignment with project conventions.
- Too little context reduces effectiveness
- Too much is noisy, costly, and can worsen behavior



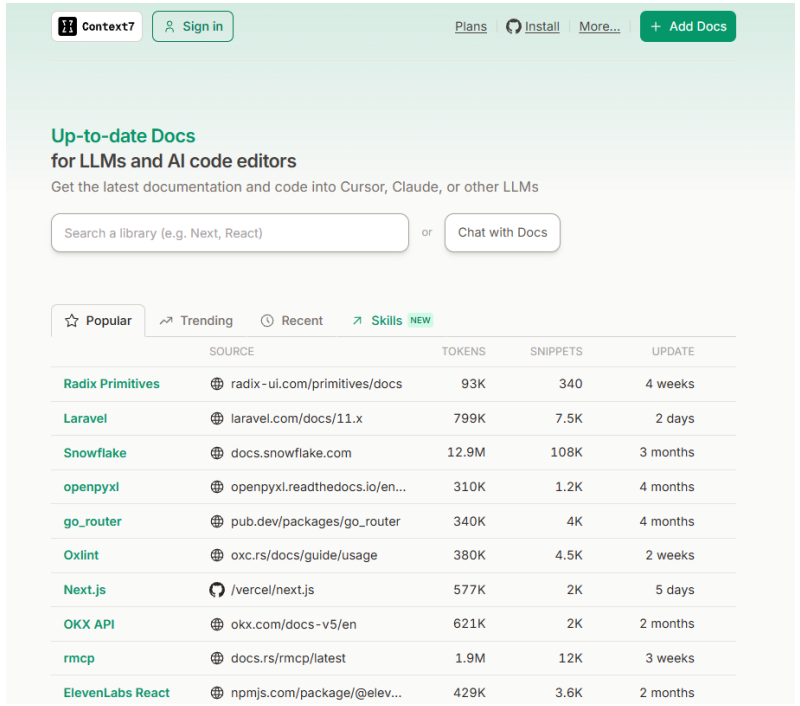
MCP (Model Context Protocol)

- An open protocol designed to enable integration between large language model (LLM) applications and external data sources, tools, or services

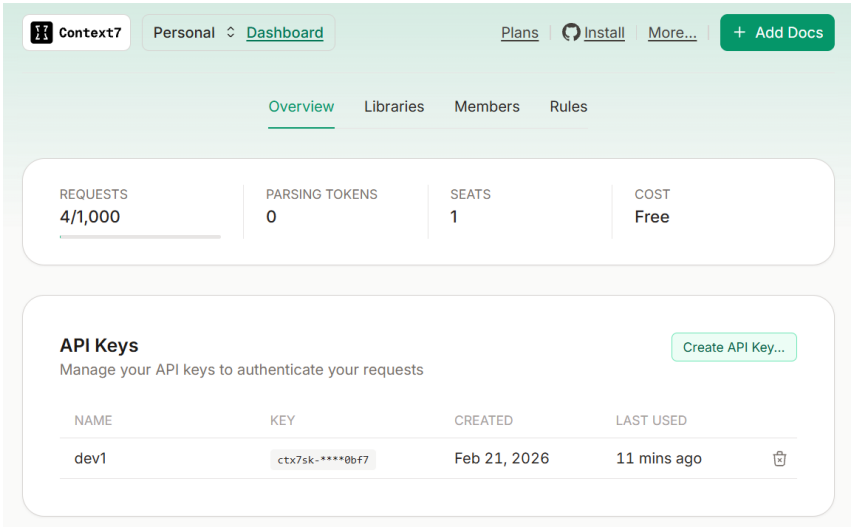


Context7: MCP developed by Upstash

- Problem: outdated or hallucinated code suggestions by coding agents
- Fetches up-to-date, version-specific documentation and code examples directly from official sources and injects them into your AI assistant's context window in real-time

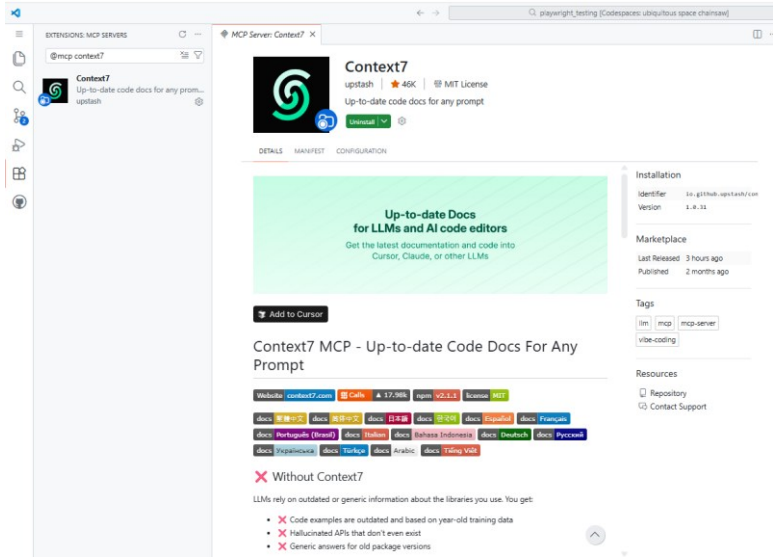


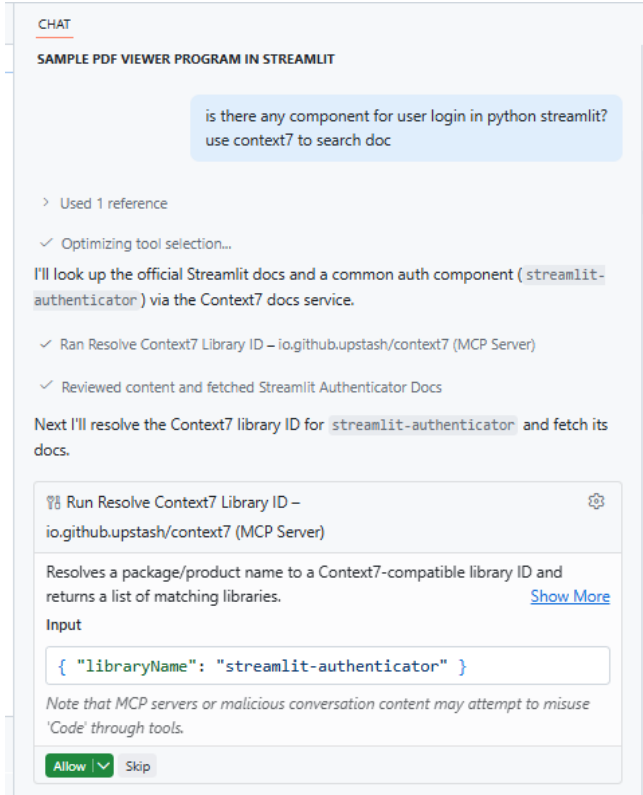
Get an API key from the context7 website



https://context7.com

Context7 MCP in VS Code






```
{
    "servers": {
        "io.github.upstash/context7": {
            "type": "stdio",
            "command": "npx",
            "args": [
                "@upstash/context7-mcp@1.0.31"
            ],
            "env": {
                "CONTEXT7_API_KEY": "${input:CONTEXT7_API_KEY}"
            },
            "gallery": "https://api.mcp.github.com",
            "version": "1.0.31"
        }
    },
    "inputs": [
        {
            "id": "CONTEXT7_API_KEY",
            "type": "promptString",
            "description": "API key for authentication",
            "password": true
        }
    ]
}
```

Agent instruction files

- A special Markdown file has been added to your project's repository
- Coding agents automatically read it each time, using the supplied context to generate better, more relevant code suggestions.
- Let AI understand your project's goals, tech stack, and conventions.
  - Produce code that matches your project's style and requirements.
  - Less likely to make errors (E.g., by using the wrong framework or library, rewrite code which already exists somewhere in your repository)

Tool	Rules
GitHub Copilot	.github/copilot-instructions.md
Cursor	.cursor/rules/*.mdc or .cursorrules
Claude Code	CLAUDE.md

Example: agent instruction files

- Project Overview
  - Briefly describe what the project or app does.
  - State its purpose, main audience, and key features.
- Tech Stack
  - List the backend and frontend technologies.
  - Identify any frameworks, libraries, databases, APIs, and testing tools.
  - Add short notes on how each is being used if relevant.
- Coding Guidelines
  - Outline the key coding standards and practices expected.
  - Include things like style (e.g., tabs vs. spaces), required tests, API design, and security practices.
  - Note any file/folder-specific rules or conventions.
- Project Structure
  - Show the main project folders and their purposes.
  - Include subfolder explanations for things like frontend, backend, tests, scripts, docs, etc.
  - Help Copilot understand where to find code, tests, and resources.
- Resources and Tools
  - List helpful scripts or automation resources (e.g., for setup, testing, deployment).
  - Reference any custom tools, available factories, or services used in development.
  - Mention other files (e.g., README, configuration, or scripts) that may help Copilot work more efficiently.

Example: https://github.blog/ai-and-ml/github-copilot/5-tips-for-writing-better-custom-instructions-for-copilot

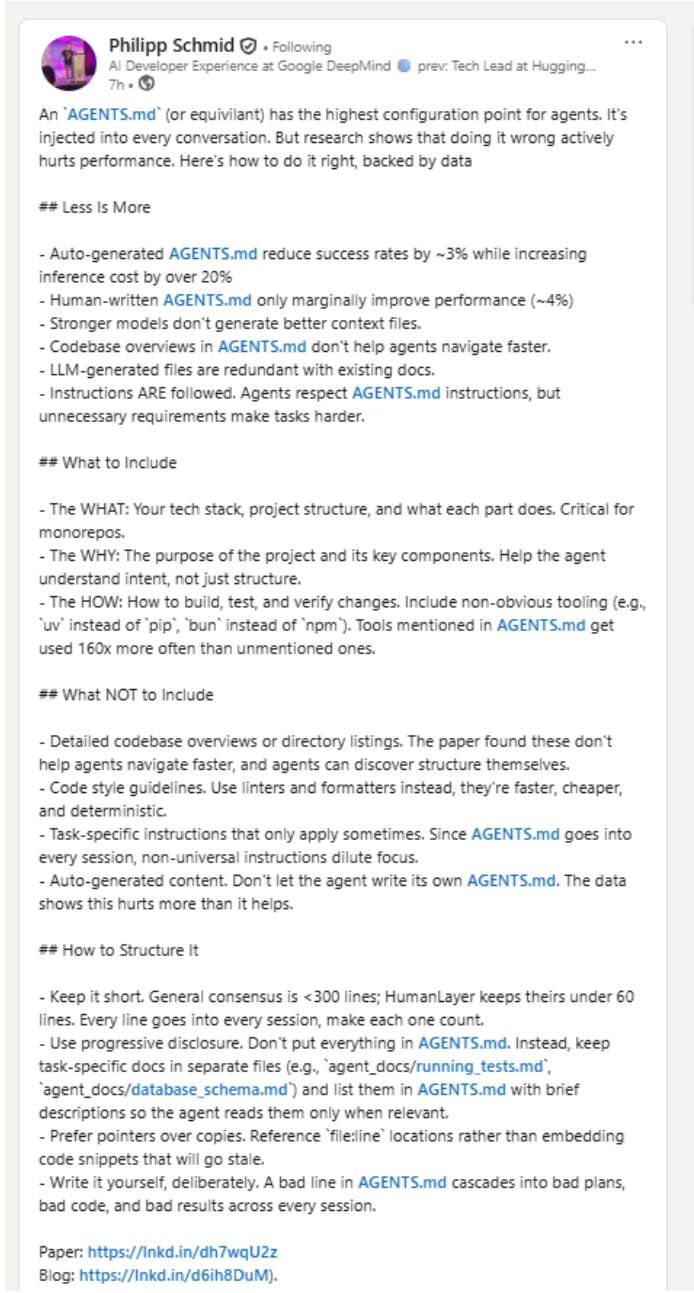
Exercise

- Create an empty repository in GitHub.
- Write a copilot instruction file (**.github/copilot-instructions.md**) with the following rules
  - Before implementing any code, ask *3* clarifying questions to make the requirements more complete
  - Based on the user requirement, create a SPEC.MD to write a detailed specification and ask user for comments
  - Based on the updated requirement, create the code and ask the user for a comment.
  - When the user provides feedback about the app, modify the SPEC.MD before making changes to the code.
  - Record in a file "LESSONS_LEARNT.MD" to record the user preference and mistakes you have made, and how you can do better next time
- Use an AI coding agent (e.g. gpt5-mini) to experiment with developing a simple app

Effective AI Instruction Files

- Auto-generated AGENTS.md reduces success rates by ~3% while increasing inference cost by over 20%
- Human-written AGENTS.md only marginally improves performance (~4%)
  LLM-generated files are redundant with existing docs.
- Agents respect AGENTS.md instructions, but unnecessary requirements make tasks harder.
- What to include
  - **WHAT:** Your tech stack, project structure, and what each part does. Critical for monorepos.
  - **WHY:** The purpose of the project and its key components. Help the agent understand intent, not just structure.
  - **HOW:** How to build, test, and verify changes. Include non-obvious tooling (e.g., `uv` instead of `pip`, `bun` instead of `npm`).
- What NOT to Include
  - Detailed codebase overviews or directory listings. The paper found that these don't help agents navigate faster, and agents can discover structure themselves.
  - Task-specific instructions that only apply sometimes.
    - Since AGENTS.md goes into every session, non-universal instructions dilute focus.
  - Auto-generated content: Don't let the agent write its own AGENTS.md. The data shows this hurts more than it helps.

Paper: Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?
Blog: https://lnkd.in/d6ih8DuM



Philipp Schmid • Following
AI Developer Experience at Google DeepMind prev: Tech Lead at Hugging...
7h •

An `AGENTS.md` (or equivilant) has the highest configuration point for agents. It's injected into every conversation. But research shows that doing it wrong actively hurts performance. Here's how to do it right, backed by data

Less Is More

- Auto-generated AGENTS.md reduce success rates by ~3% while increasing inference cost by over 20%
- Human-written AGENTS.md only marginally improve performance (~4%)
- Stronger models don't generate better context files.
- Codebase overviews in AGENTS.md don't help agents navigate faster.
- LLM-generated files are redundant with existing docs.
- Instructions ARE followed. Agents respect AGENTS.md instructions, but unnecessary requirements make tasks harder.

What to Include

- The WHAT: Your tech stack, project structure, and what each part does. Critical for monorepos.
- The WHY: The purpose of the project and its key components. Help the agent understand intent, not just structure.
- The HOW: How to build, test, and verify changes. Include non-obvious tooling (e.g., `uv` instead of `pip`, `bun` instead of `npm`). Tools mentioned in AGENTS.md get used 160x more often than unmentioned ones.

What NOT to Include

- Detailed codebase overviews or directory listings. The paper found these don't help agents navigate faster, and agents can discover structure themselves.
- Code style guidelines. Use linters and formatters instead, they're faster, cheaper, and deterministic.
- Task-specific instructions that only apply sometimes. Since AGENTS.md goes into every session, non-universal instructions dilute focus.
- Auto-generated content. Don't let the agent write its own AGENTS.md. The data shows this hurts more than it helps.

How to Structure It

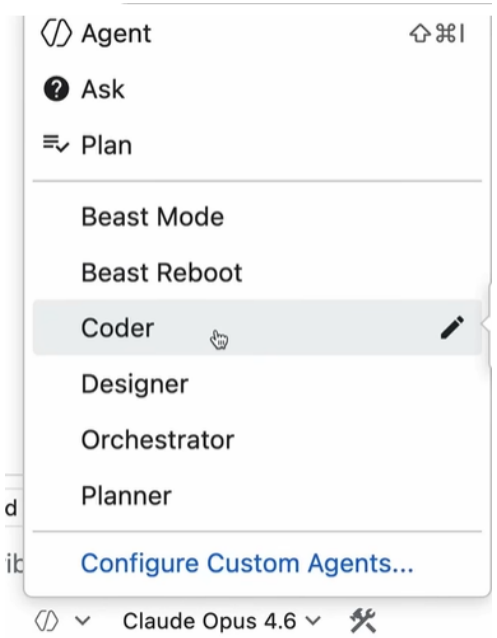
- Keep it short. General consensus is <300 lines; HumanLayer keeps theirs under 60 lines. Every line goes into every session, make each one count.
- Use progressive disclosure. Don't put everything in AGENTS.md. Instead, keep task-specific docs in separate files (e.g., `agent_docs/running_tests.md`, `agent_docs/database_schema.md`) and list them in AGENTS.md with brief descriptions so the agent reads them only when relevant.
- Prefer pointers over copies. Reference `file:line` locations rather than embedding code snippets that will go stale.
- Write it yourself, deliberately. A bad line in AGENTS.md cascades into bad plans, bad code, and bad results across every session.

Paper: https://lnkd.in/dh7wqU2z
Blog: https://lnkd.in/d6ih8DuM).

Source: Linkedin

Custom Agents

- Custom agents in VS Code are user-defined AI configurations for GitHub Copilot that let you tailor the AI's behavior to specific roles or tasks
  - Stored as .agent.md files in the .github/agents folde
- Examples:
  - Planners, coders, and designers:
    - Potentially using different AI models best suited for that role.
  - The orchestrator agent delegates work to various sub-agents.
    - Isolated context windows for sub-agents
    - Preventing the main context from becoming overloaded
    - Enable parallel processing of tasks.



Video: After This Video, You'll Actually Understand Agent Orchestration

Example

```
---
name: Orchestrator
description: Sonnet, Codex, Gemini
model: Claude Sonnet 4.5 (copilot)
tools: ['read/readFile', 'agent', 'memory']
---

You are a project orchestrator. You break down complex requests into
tasks and delegate to specialist subagents. You coordinate work but NEVER
implement anything yourself.

## Agents

These are the only agents you can call. Each has a specific role:

- **Planner** — Creates implementation strategies and technical plans
- **Coder** — Writes code, fixes bugs, implements logic
- **Designer** — Creates UI/UX, styling, visual design

## Execution Model

You MUST follow this structured execution pattern:

### Step 1: Get the Plan
Call the Planner agent with the user's request. The Planner will return
implementation steps.

### Step 2: Parse Into Phases
......
```

Example: [Ultralight Orchestration]

```
---
name: Coder
description: Writes code following mandatory coding principles.
model: Claude Opus 4.6 (copilot)
tools: ['vscode', 'execute', 'read', 'agent', 'context7/*', 'github/*',
'edit', 'search', 'web', 'memory', 'todo']
---

ALWAYS use #context7 MCP Server to read relevant documentation. Do this
every time you are working with a language, framework, library etc. Never
assume that you know the answer as these things change frequently. Your
training date is in the past so your knowledge is likely out of date,
even if it is a technology you are familiar with.

## Mandatory Coding Principles

These coding principles are mandatory:

1. Structure
- Use a consistent, predictable project layout.
- Group code by feature/screen; keep shared utilities minimal.
- Create simple, obvious entry points.
- Before scaffolding multiple files, identify shared structure first. Use
framework-native composition patterns (layouts, base templates,
providers, shared components) for elements that appear across pages.
Duplication that requires the same fix in multiple places is a code
smell, not a pattern to preserve.

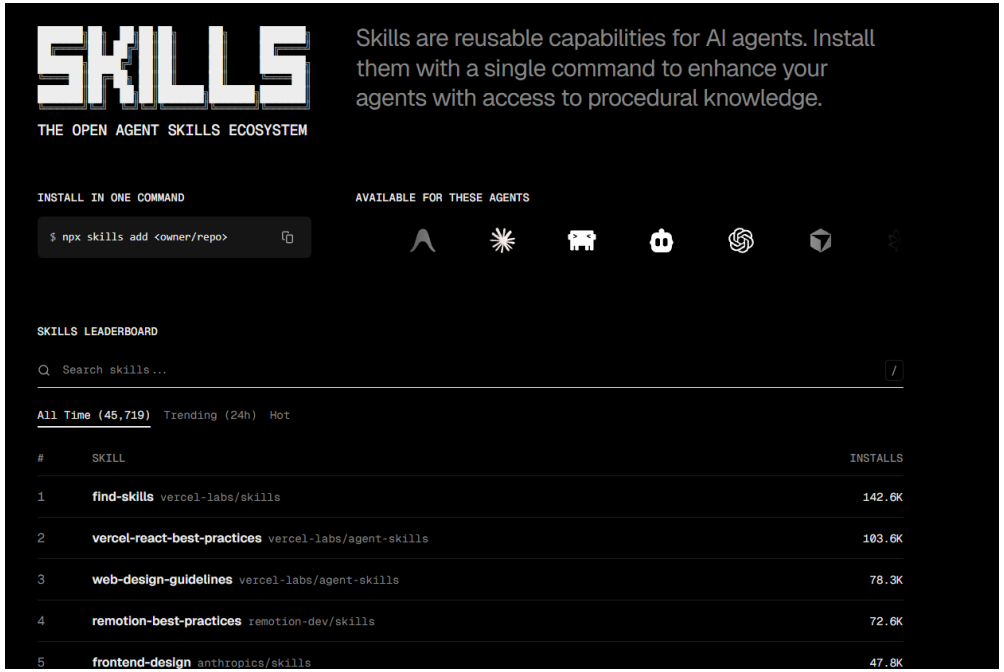
2. Architecture
- Prefer flat, explicit code over abstractions or deep hierarchies.
- Avoid clever patterns, metaprogramming, and unnecessary indirection.
- Minimize coupling so files can be safely regenerated.

3. Functions and Modules
- Keep control flow linear and simple.
- Use small-to-medium functions; avoid deeply nested logic.
- Pass state explicitly; avoid globals.

......
```

Agent Skills

- Agent skills are modular folders containing instructions, scripts, resources, and a required SKILL.md file with YAML metadata (like name and description)
- AI agents, such as GitHub Copilot in VSCode, can discover, match to user queries via semantic descriptions, and load on-demand to perform specialized tasks efficiently without bloating context.
- Video: The complete guide to Agent Skills



https://skills.sh

SKILLS.md

```
---

name: next-best-practices

description: Next.js best practices - file conventions, RSC
boundaries, data patterns, async APIs, metadata, error handling,
route handlers, image/font optimization, bundling

user-invocable: false

---

# Next.js Best Practices

Apply these rules when writing or reviewing Next.js code.

## File Conventions

See [file-conventions.md](./file-conventions.md) for:

- Project structure and special files
- Route segments (dynamic, catch-all, groups)
- Parallel and intercepting routes
- Middleware rename in v16 (middleware → proxy)

...
```

https://github.com/vercel-labs/next-skills/tree/main/skills/next-best-practices

```
---

name: pdf

description: Use this skill whenever the user wants to do anything with
PDF files. This includes reading or extracting text/tables from PDFs,
combining or merging multiple PDFs into one, splitting PDFs apart,
rotating pages, adding watermarks, creating new PDFs, filling PDF forms,
encrypting/decrypting PDFs, extracting images, and OCR on scanned PDFs to
make them searchable. If the user mentions a .pdf file or asks to produce
one, use this skill.

license: Proprietary. LICENSE.txt has complete terms

---

# PDF Processing Guide

## Overview

This guide covers essential PDF processing operations using Python
libraries and command-line tools. For advanced features, JavaScript
libraries, and detailed examples, see REFERENCE.md. If you need to fill
out a PDF form, read FORMS.md and follow its instructions.

## Quick Start


```python

from pypdf import PdfReader, PdfWriter

# Read a PDF

reader = PdfReader("document.pdf")

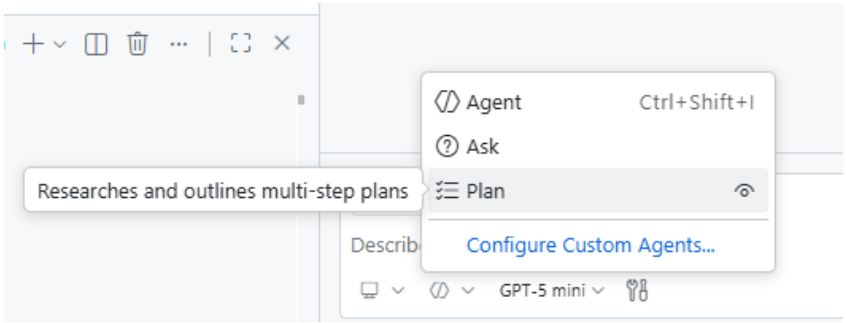
print(f"Pages: {len(reader.pages)}")

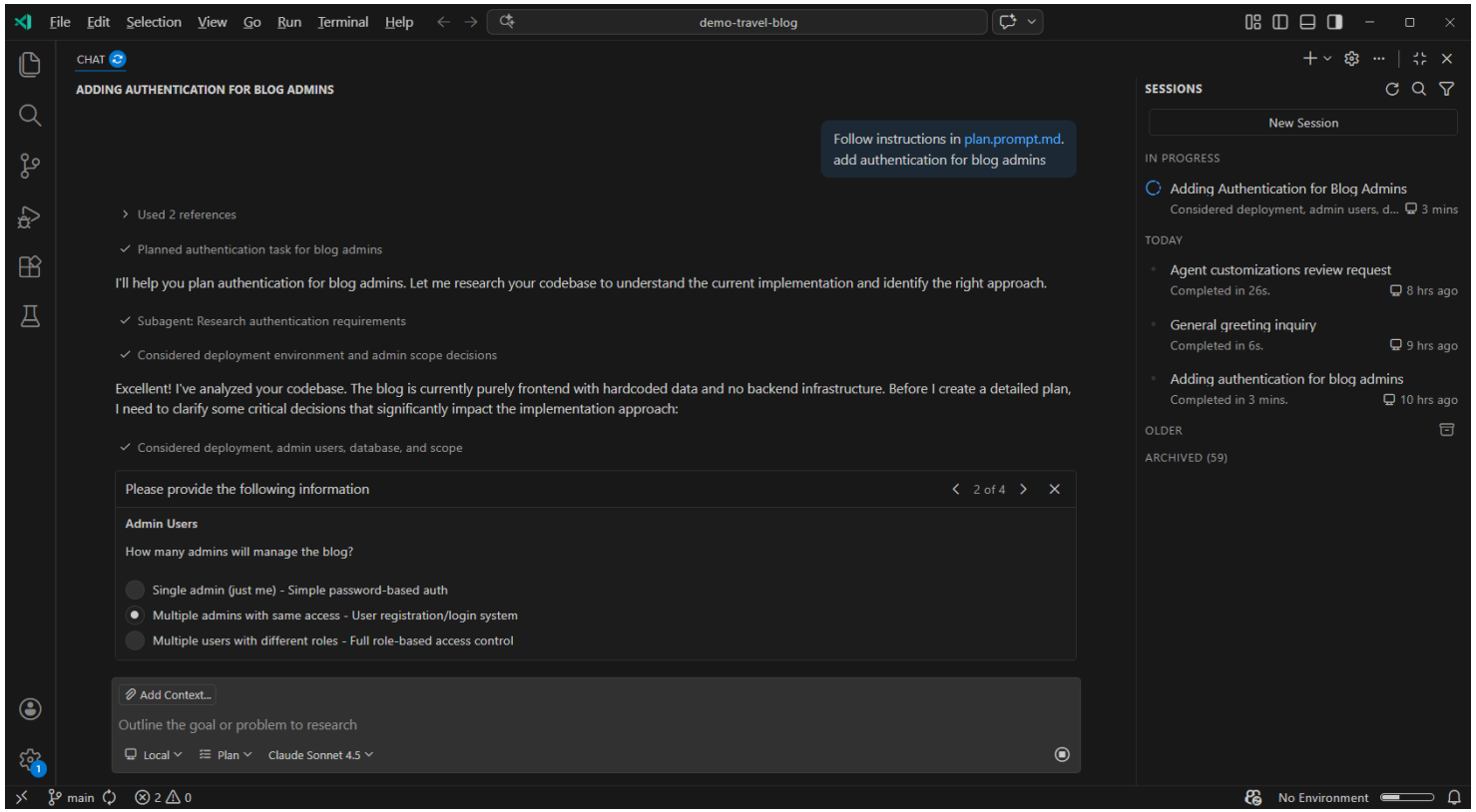
......
```

https://github.com/anthropics/skills/tree/main/skills/pdf
```

Plan mode

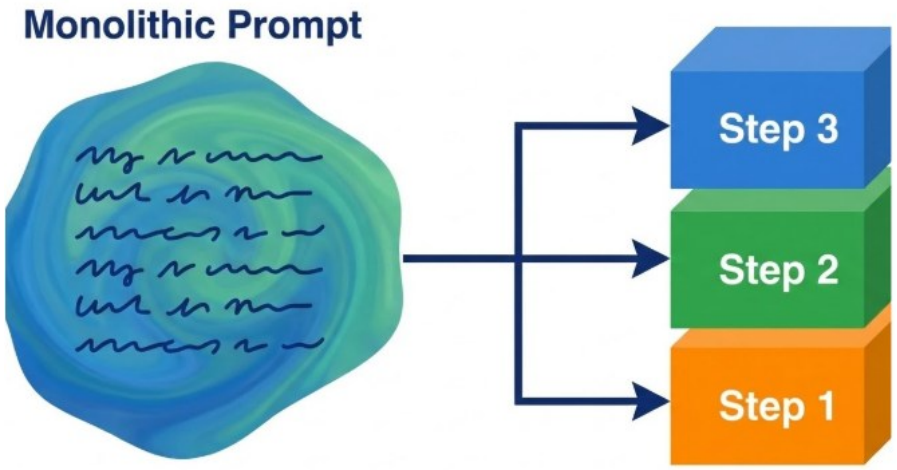
- Create a detailed implementation plan before any code is writte





Work in small, testable increments

- Large, monolithic prompts lead to inconsistency, duplication, and a "jumbled mess"
- Break work into small, bite-sized "tickets" or milestones.
  - Implement a single function, verify it with tests, and then iterate to the next step
  - Avoid a single change that touches frontend + backend + database all at once.
  - When something breaks, you can test one layer at a time and pinpoint the issue faster.



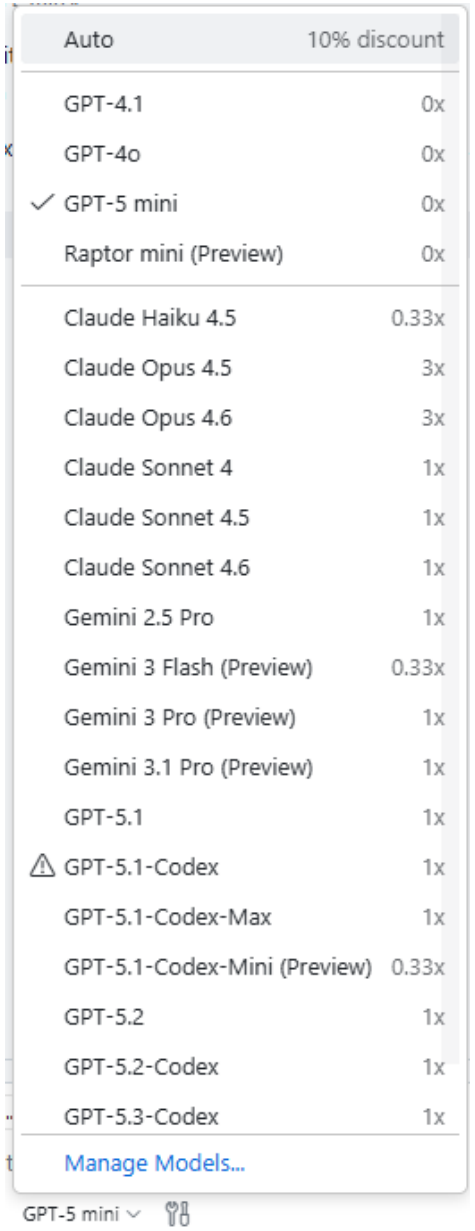
"Make me a Google Drive."

Example workflow:

1. Implement **frontend** with mocked/fake data.
2. Implement **backend REST API** and validate with a small **test client**
3. Integrate **frontend** + **backend**.
4. Add/integrate the **database**.
5. Refactor once the end-to-end path works.

Choosing the right LLM model

- Not all coding LLMs are equal
  - Each LLM model has a specific personality and strengths.
  - Use reasoning-heavy models (like Claude) for planning/architecture and high-speed models for implementation
- If one model gets stuck, swap it out for a different service to break through blind spots

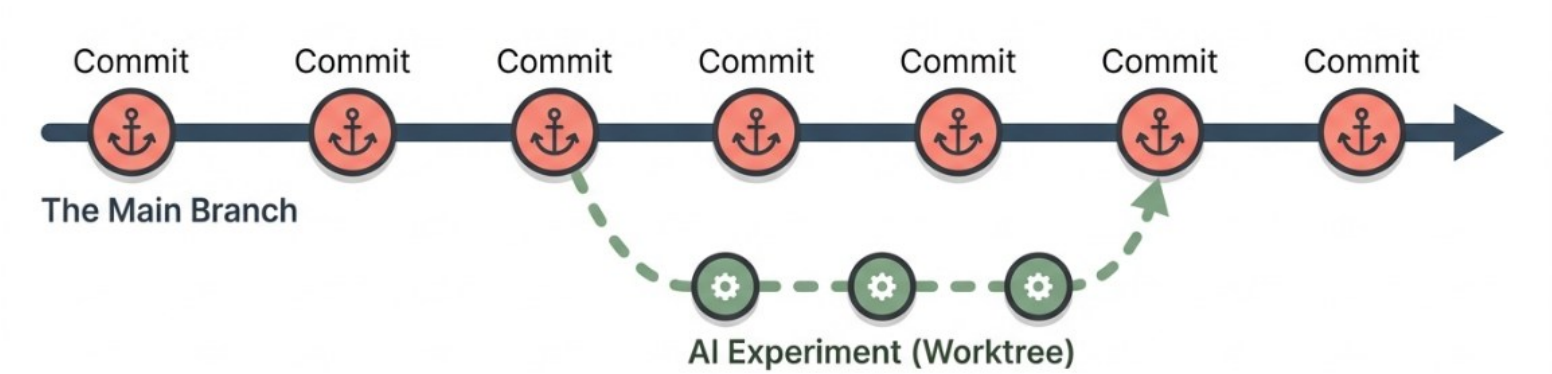


Discussion:

Which LLM model is the most useful to you? Why?

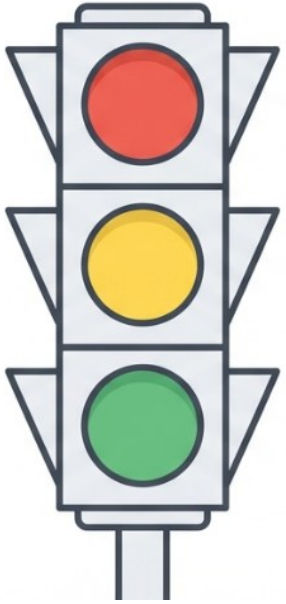
Commit Often and Use Version Control as a Safety Net

- Frequent, small commits act like "save points," allowing quick rollback when AI-generated code goes off course
  - Commits after every small task or successful automated change, more often than typical hand-coding habits.
- Uses branches/isolated git worktrees as sandboxes for separate AI experiments, allowing concurrent AI sessions without conflict.



The Traffic Light Protocol for AI Usage

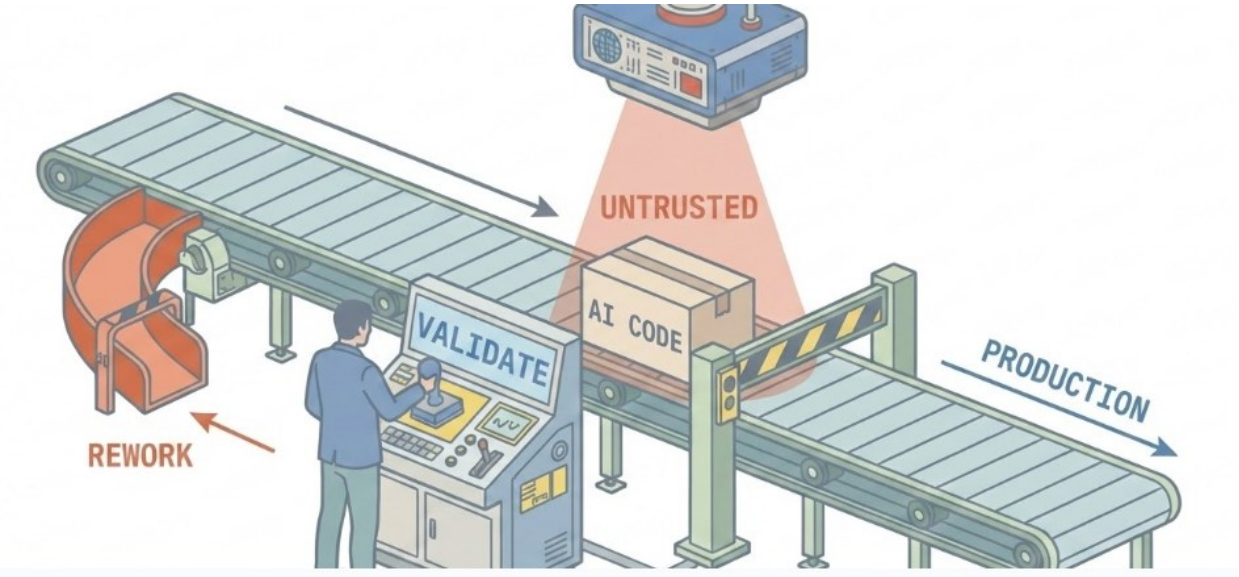
Zone	Definition	AI Permissibility	Baytech Governance Recommendation
Red Zone	Core Security, Auth, Payments, Cryptography, PII Handling	**Low** (Restricted)	**Zero Trust:** Manual coding preferred. If AI is used, it must undergo deep security audit and penetration testing.
Yellow Zone	Business Logic, API Integrations, Data Transformation, State Management	**Medium** (Assisted)	**Co-Pilot Mode:** AI suggests, Human verifies line-by-line. Strict linting required. Code must be fully understood by the reviewer.
Green Zone	Boilerplate, Unit Tests, Documentation, UI Mockups, Internal Prototypes	**High** (Autonomous)	Use "Vibe Coding" for speed. Review for style only. AI can generate 80-90% of this content.



https://www.baytechconsulting.com/blog/ai-technical-debt-how-vibe-coding-increases-tco-and-how-to-fix-it

Treat AI Code as Untrusted Input

- Adopt a ‘Zero Trust’ policy for AI-generated code.
  - Treat every AI-generated snippet as if it came from an over-confident junior intern.
  - Never merge code you haven’t read and understood.
- Engineer-in-the-Loop
  - Establish explicit policies requiring qualified human validation for all AI contributions.

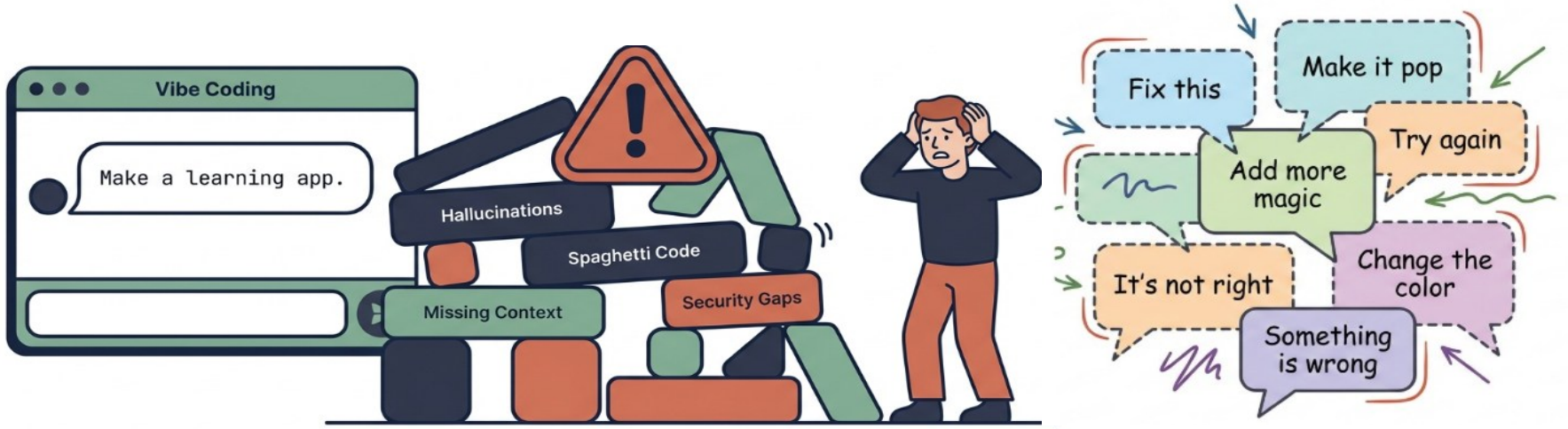


Refactoring

- AI-assisted coding will accumulate technical debt.
- As the codebase becomes larger and larger, the code becomes messy, hard to maintain/fix bugs, and add new features
- Refactoring from time to time
  - Break long functions into smaller units
  - Break large modules into cohesive modules
  - Remove/merge duplicated functionality (AI likes to generate new code rather than using existing code)
  - Make code more reusable and testable

Chatting is not engineering

- Traditional "vibe coding“ (chatting casually with AI) lacks context and foresight.
- Generates ‘house of cards’ software that works initially but is hard to maintain, debug, or scale once the context window closes.



Spec first approach

- **Mistake**: Diving straight into code generation with vague prompts.
- Spend some time to iterate on the specification with the AI before writing any code
  - Requirements (edge cases and user flows), architecture (tech stack), data models (schema), and testing strategy

Specification Driven Development (SDD)

- A new software engineering methodology where formal, detailed specifications serve as the "executable blueprints" and primary source of truth for autonomous AI agents.
- In SDD, the code is considered a "derived artifact" or an "intermediate artifact," while the specification is the actual product (source of truth)
  - Developers define the what (requirements) and why (intent) in a structured format;
  - The AI handles the how (implementation).

Key Components of the Specification

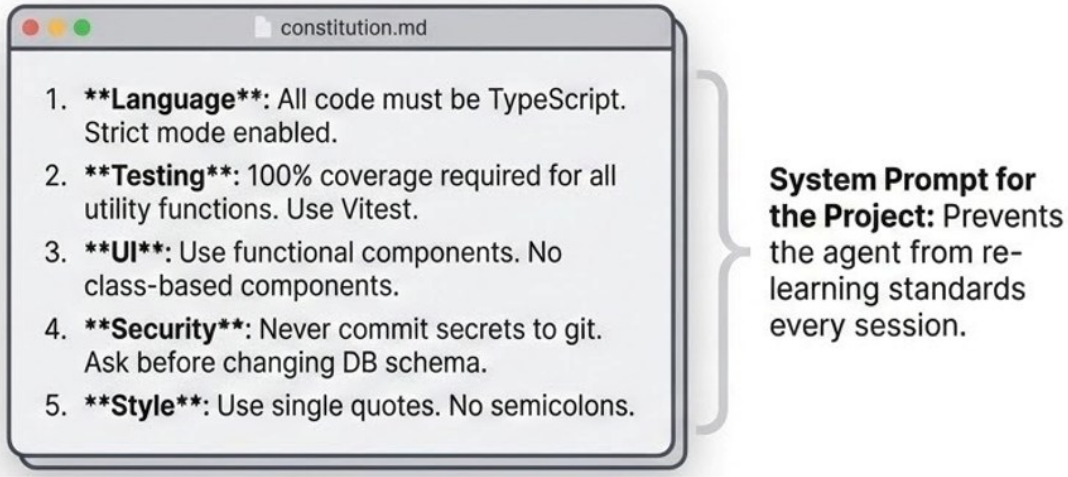
Tools like GitHub Spec Kit and AWS Kiro operationalize SDD through a structured lifecycle that guides the AI agent

Key components:

- The Constitution (constitution.md)
- Functional Spec (spec.md)
- Technical Plan (plan.md)
- Context Files
  - Files like AGENTS.md or SKILL.md provides persistent instructions and "skills" to the agent
  - Acting like a "README" for AI agents

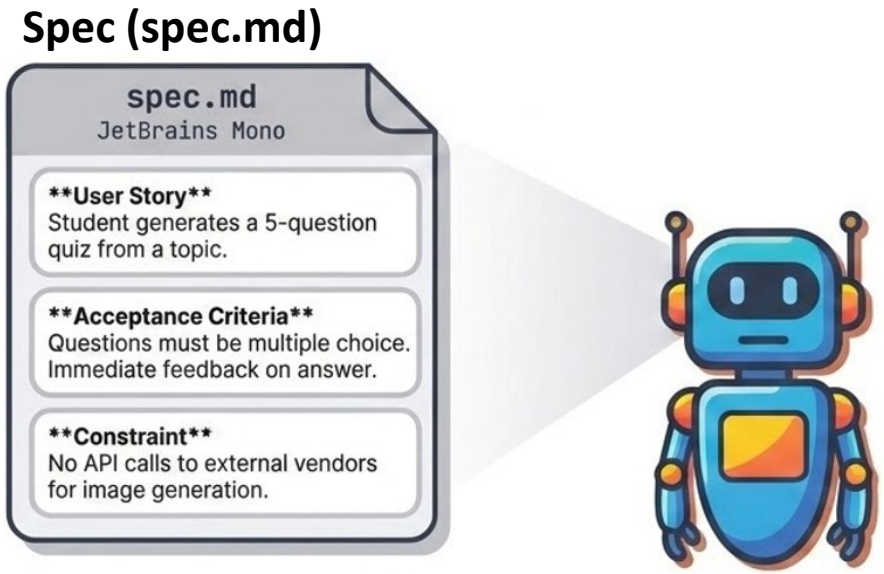
The Constitution (constitution.md)

- A governance document defining non-negotiable principles
  - Tech stack (e.g. React/Tailwind CSS), always run tests before committing, use TypeScript strict mode, never commit secrets, etc
  - This ensures the AI follows team standards across every task



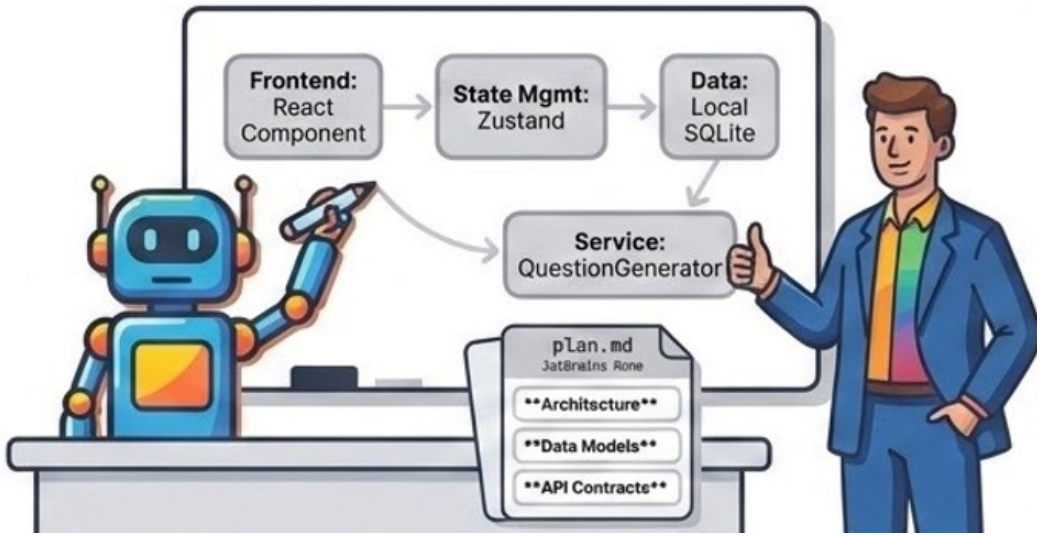
Phase 1: Specify (The “What”)

- The developer defines the user stories, acceptance criteria, edge cases, error handling requirements, and success metrics.
- Excludes technical decision-making to focus purely on user outcomes and business value



Phase 2: Plan (The "How")

- The Agent consumes the Spec and proposes a Technical Plan.
  - It decides the architecture, data models, and API contracts *before* writing a single line of code.
- The developer reviews this plan to ensure it matches engineering standards and the project Constitution.



Phase 3: Atomic Task Generation

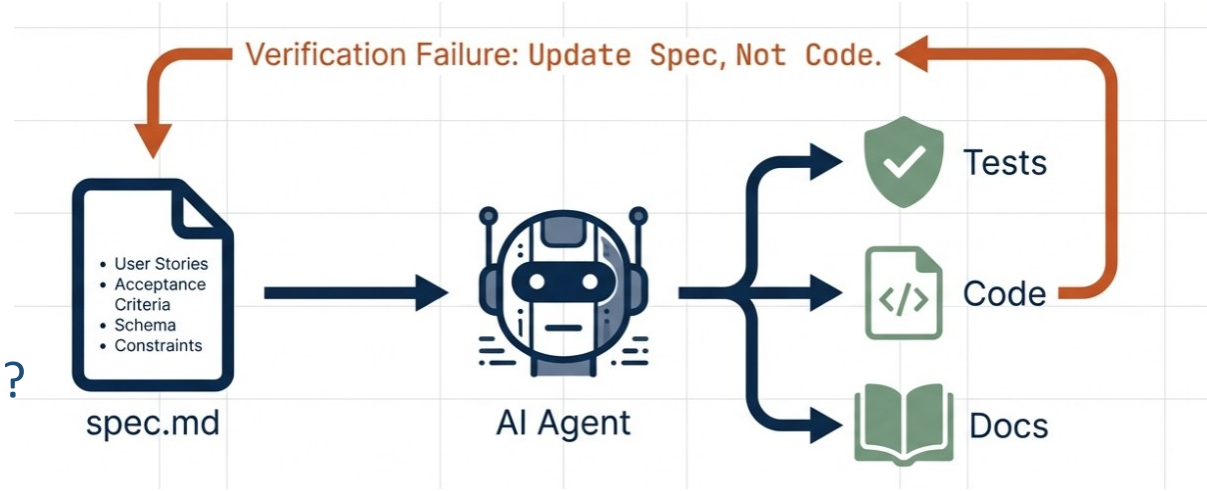
- The Plan is broken into small, testable tasks.
- The AI doesn't try to build the whole app at once; it executes one atomic task at a time.
- This allows for checkpoints where the human can verify progress against the Spec.

Phase 4: Implement (Agent execution)

- The Agent implements the code task-by-task.
- Because it follows a rigid **Plan** and **Spec**, the code is consistent.
- The developer's role shifts from writing syntax to verifying the output against the acceptance criteria defined in Phase 1.

Specification is the source of truth

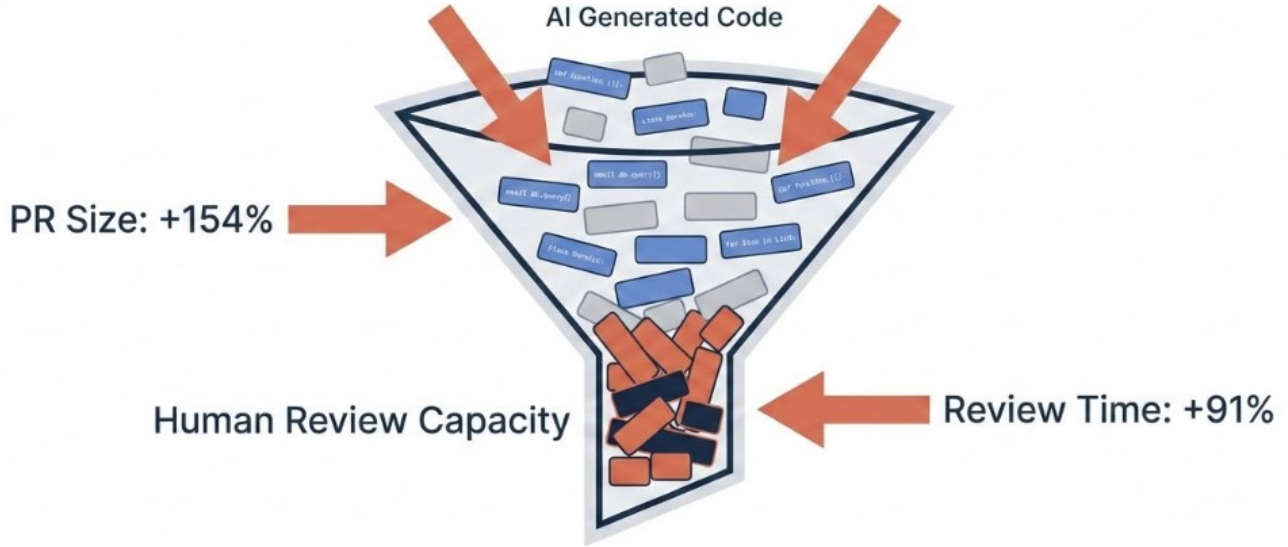
- **Spec Drift:** If an AI agent makes code changes without checking the spec, the code and the documentation diverge ("drift") from the spec.
- When a bug arises, it is treated as a missing requirement.
  - The **Spec** is updated to be more explicit to fix the issue permanently.
  - This preserves context for future agents and prevents regressions that often happen when humans manually patch code.
- When adding features, we don't just patch the code. We update the **Spec** to include new logic
  - The AI identifies the **delta**, updates the **Plan** to include new data models and services, generates new **Tasks**, and implements changes safely without breaking existing logic.



Discussion: Do you think this approach works well? What's the challenge?

MERT Study (2025)

- Randomized controlled trial with experienced developers across 246 real-world coding tasks.
- While developers expected AI to speed them up by 24%, developers take 19% longer to complete issues when using AI tools
  - Code review times have increased 91% for teams heavily adopting AI.
  - Pull request sizes have grown 154
- While we feel productivity by pressing tab and accepting the suggestion, we are also accumulating code that we don't fully understand.



https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study

The Six Levels of AI-Assisted Software Product Development

Dan Shapiro published a framework that maps AI coding adoption like the levels of driving automation:

[Source]

Level 0: Spicy Autocomplete

- Start typing. The AI suggests the next few characters or lines to save a few seconds. It's convenient. But we're in the same old paradigm. At this level, the human is the "Code Master". The code unmistakably belongs to the human. This is manual labor.

Level 1: The Coding Intern

- The human gives the AI discrete, atomic tasks within a file. They ask it to write a function, add unit tests, clean up code, etc. At this level, the human is a "Task Master". They write the "important" stuff and use the AI for small, scoped chores.

Level 2: The Junior Developer

- The AI has multi-file awareness and tools are "AI-native". This allows larger tasks to be handed over to the AI. At this level, developers feel a massive flow state and productivity boost.

Level 3: The Developer

- The AI acts as the Senior Developer, running multiple tasks and tabs in parallel. At this level, the human almost stops writing code. They review PRs and diffs, spending most of their time ensuring the AI hasn't veered off course.

Level 4: The Engineering Team

- At this level, the human writes a spec, argues with the AI about it, and leaves for 12+ hours while it builds and tests. They effectively become a Product Owner / Manager, focusing on the "what" and "why," checking whether tests pass and whether the value is there.

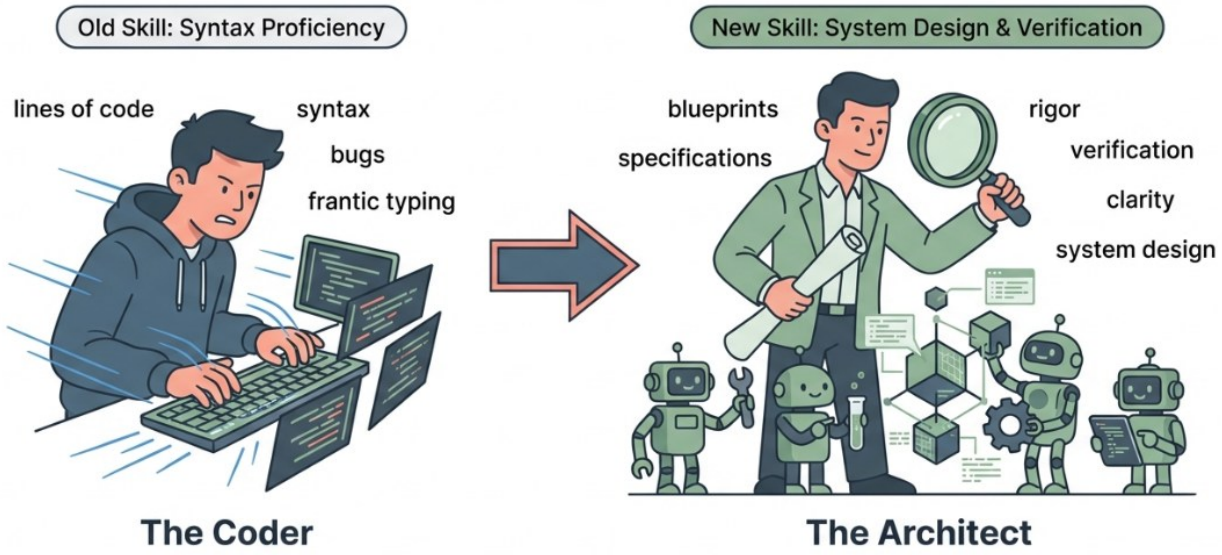
Level 5: The Dark Factory

- At this level, the AI is a sophisticated system that turns goals and high-level ideas into production-ready software autonomously. Humans are "not welcome" in the implementation loop. Instead, they set the vision and design the feedback loops that allow the system to improve continuously.

Discussion:
Read this [LinkedIn] post. What are the pros and cons of each level? Which level do you think works best for software development today?

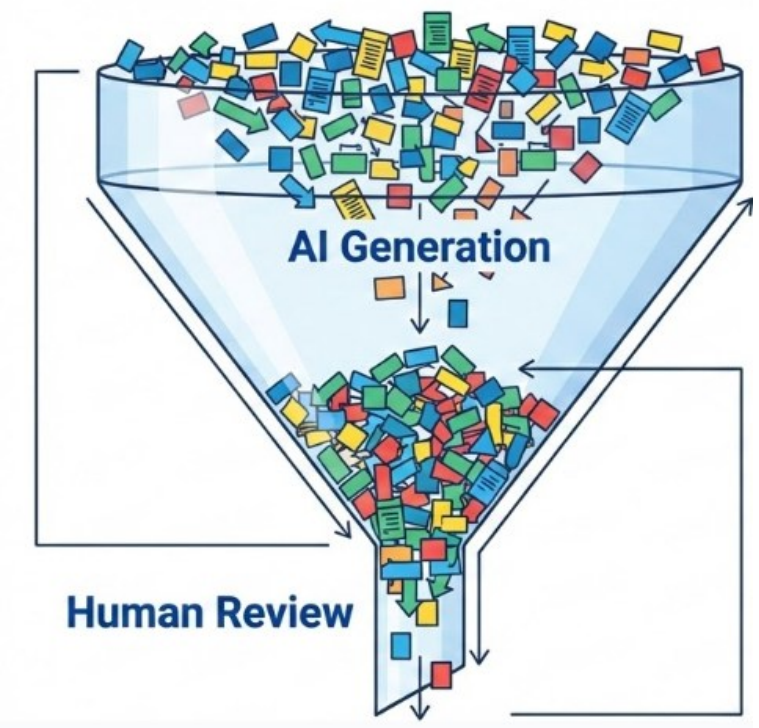
Rise of Architect

- The developer's role is evolving. Instead of being judged by lines of code written
- You are judged by:
  - The clarity of your **specifications**
  - The rigor of your **verification**



The new bottleneck: Review Capacity

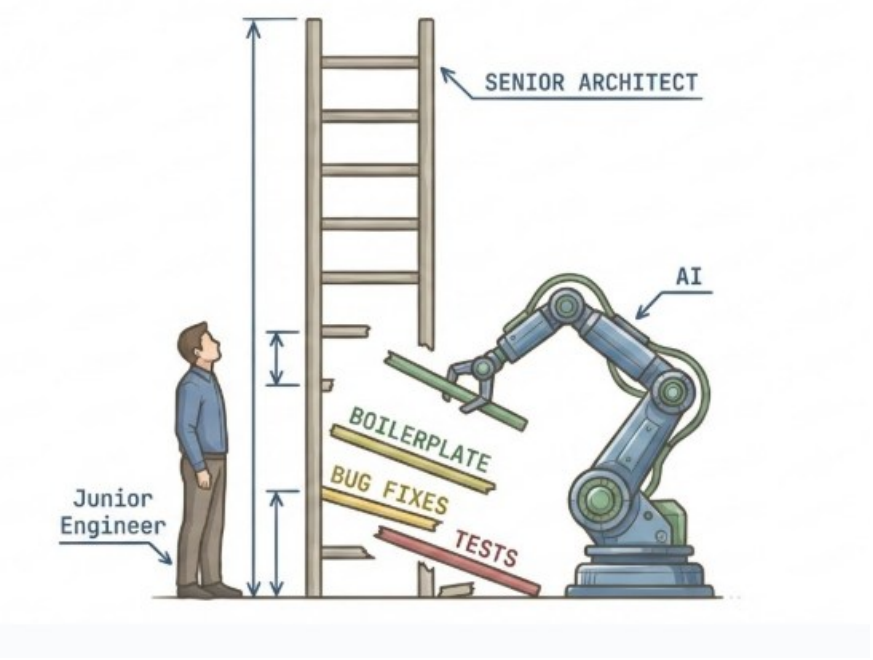
- Code generation is easy
- Verification is the differentiator.
  - Writing good tests (unit/integration/e2e)
  - Reading code critically
  - Debugging systematically



- We optimized to produce less code.
- Now we must optimize for verifying abundant code rigorously.

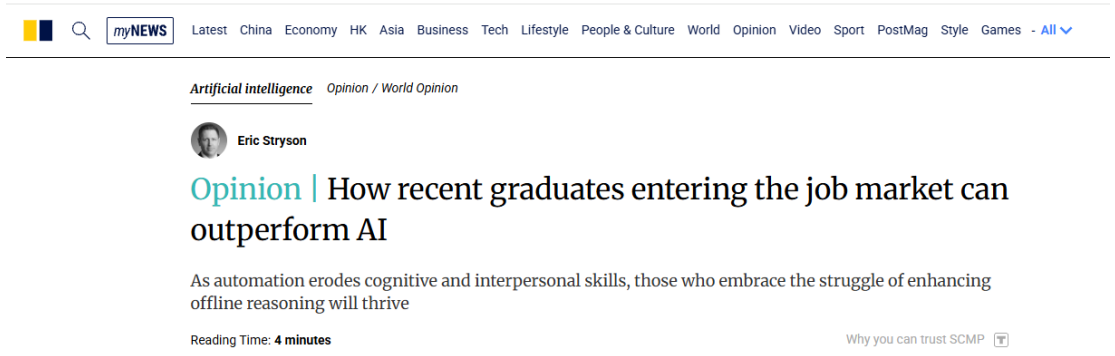
Junior Developer Crisis

- AI is automating the learning path.
  - Current AI was trained on code written by humans with a deep understanding.
- The tasks AI handles best (boilerplate, simple bugs, test generation) are precisely the training grounds for junior engineers.
- By letting juniors skip the 'struggle,' we remove the opportunity to learn from first principles.
  - If we stop hiring or training juniors now, we face a crisis where no one understands the systems deeply enough to debug them when AI fails.



Video: Vibe Coding is a Trap (What Senior Devs See That You Don't)
https://www.youtube.com/watch?v=ya6520zh4pQ

How recent graduate entering job market can outperform AI



myNEWS Latest China Economy HK Asia Business Tech Lifestyle People & Culture World Opinion Video Sport PostMag Style Games - All

Artificial intelligence *Opinion / World Opinion*

Eric Stryson

Opinion | How recent graduates entering the job market can outperform AI

As automation erodes cognitive and interpersonal skills, those who embrace the struggle of enhancing offline reasoning will thrive

Reading Time: **4 minutes**

Why you can trust SCMP

The numbers are starting to confirm what we suspected. Hong Kong graduates in 2025 found 55 per cent fewer job opportunities than the year before. Over 12 per cent of those aged 20 to 24 were unemployed in December, the second-highest figure on record.

The temptation grows when AI assistants live in every app. When chatbots can produce 20-page reports in minutes, they foster impatience, shallow understanding and an unwillingness to do extra work.

Hiring managers make decisions based on economics, not social good. They need to be convinced that a young human worker is a better investment than a machine. Many employers are reluctant to bet on people with no practical experience and no domain knowledge.

Management may prefer to upskill experienced staff to use AI to do more of what they already do. A perceived preference for job-hopping among younger workers only compounds the disincentive. This conforms to our experience as a small company paying high Hong Kong rents.

The "productive struggle" of thinking through hard problems, failing, retrying and eventually making breakthroughs has always been at the core of genuine learning. That this struggle is increasingly bypassed is what keeps educators and developmental psychologists awake at night. It's also what troubles managers when junior staff expect work to be easy. The friction of navigating daily life, of developing street smarts, is being removed for AI natives.

Bolder still, regulators should introduce cognitive capability standards in professional licensing and accreditation. If you want to practise law or accounting, you must show you can reason through a complex problem without an algorithm. This would create artificial market demand for human skills at risk of atrophy.

Source: SCMP (15 Feb 2026)

Continuously Learn and Adapt

- When AI fails on edge cases, engineers who have lost fundamental skills cannot recover
- Deepened knowledge of languages, frameworks, and design patterns through interacting with AI.
- Use AI in “ask mode” to learn
  - Request explanations, alternatives, and failure modes

Practicing regularly without AI

- The tasks AI handles, boilerplate, simple bugs, test generation, can become learning exercises rather than production shortcuts.
- Use AI as an explanation engine: have juniors ask AI to explain generated code line by line, then explain it back to a senior.
- Understanding why code works matters more than producing it
- Train critical evaluation through AI code review
  - Review AI-generated code specifically to find flaws, security issues, and architectural violations.

References

- My LLM coding workflow going into 2026
  - https://addyosmani.com/blog/ai-coding-workflow/